Memory Management Techniques - C Programs

1. First Fit

```
#include <stdio.h>

int main() {
    int b[10], p[10], i, j, nb, np;

    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter block sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d", &b[i]);

    printf("Enter process sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d", &p[i]);

    for(i=0;i<np;i++) {
        for(j=0;j<nb;j++) {
            if(b[j] >= p[i]) {
                printf("Process %d allocated to block %d\n", i+1, j+1);
                b[j] -= p[i];
                break;
            }
        }
    }

    return 0;
}
```

Sample Input / Output

```
Sample Input:
Enter number of blocks: 3
Enter number of processes: 3
Enter block sizes:
100 500 200
Enter process sizes:
90 150 100

Sample Output:
Process 1 allocated to block 1
Process 2 allocated to block 2
Process 3 allocated to block 2
```

2. Best Fit

```
#include <stdio.h>

int main() {
    int b[10], p[10], i, j, nb, np, best;

    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter block sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d", &b[i]);

    printf("Enter process sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d", &p[i]);

    for(i=0;i<np;i++) {
        best = -1;

        for(j=0;j<nb;j++) {
```

```
            if(b[j] >= p[i]) {
                if(best == -1 || b[j] < b[best])
                    best = j;
            }
        }

        if(best != -1) {
            printf("Process %d allocated to block %d\n", i+1, best+1);
            b[best] -= p[i];
        }
    }

    return 0;
}
```

Sample Input / Output

```
Sample Output:
Process 1 allocated to block 1
Process 2 allocated to block 3
Process 3 allocated to block 2
```

3. Worst Fit

```
#include <stdio.h>

int main() {
    int b[10], p[10], i, j, nb, np, worst;

    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter block sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d", &b[i]);

    printf("Enter process sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d", &p[i]);

    for(i=0;i<np;i++) {
        worst = -1;

        for(j=0;j<nb;j++) {
            if(b[j] >= p[i]) {
                if(worst == -1 || b[j] > b[worst])
                    worst = j;
            }
        }

        if(worst != -1) {
            printf("Process %d allocated to block %d\n", i+1, worst+1);
            b[worst] -= p[i];
        }
    }

    return 0;
}
```

Sample Input / Output

```
Sample Output:
Process 1 allocated to block 2
Process 2 allocated to block 2
Process 3 allocated to block 3
```

4. Next Fit

```
#include <stdio.h>

int main() {
    int b[10], p[10], i, j, nb, np, last = 0;
```

```
    printf("Enter number of blocks: ");
    scanf("%d", &nb);

    printf("Enter number of processes: ");
    scanf("%d", &np);

    printf("Enter block sizes:\n");
    for(i=0;i<nb;i++)
        scanf("%d", &b[i]);

    printf("Enter process sizes:\n");
    for(i=0;i<np;i++)
        scanf("%d", &p[i]);

    for(i=0;i<np;i++) {
        int allocated = 0;

        for(j=0;j<nb;j++) {
            int index = (last + j) % nb;

            if(b[index] >= p[i]) {
                printf("Process %d allocated to block %d\n", i+1, index+1);
                b[index] -= p[i];
                last = index;
                allocated = 1;
                break;
            }
        }

        if(!allocated)
            printf("Process %d not allocated\n", i+1);
    }

    return 0;
}
```

Sample Input / Output

```
Sample Output:
Process 1 allocated to block 1
Process 2 allocated to block 2
Process 3 allocated to block 2
```